

[개발자 포트폴리오 웹 서비스]

소프트웨어 설계서

- 양지우 -

깃허브 링크: <https://github.com/qkwltkwkd1/GitHub-Sync.git>

문서번호 : [양지우]요구사항분석서_260523_Doc-005

소 속 : 소프트웨어학과

학 번 : 2024125037

팀 원 : 양지우

교 수 : 최차봉 교수님

제/개정 이력

버전	날짜	작성자 성명	제/개정사항	비 고
v1.0.0	2026-03-16	양지우	프로젝트 정의서 및 초기 폴더 구조 생성	
v1.1.0	2026-03-29	양지우	프로젝트 관리 계획서 초안 작성 및 README 업데이트	
v1.2.0	2026-04-11	양지우	요구사항 반영 프로젝트 관리 계획서 최종본 작성	
v1.2.1	2026-04-11	양지우	프로젝트 관리 계획서 수정	
v1.3.0	2026-04-25	양지우	요구사항 정의서 작성	
v1.4.0	2026-05-03	양지우	요구사항 분석서 작성	
v1.4.1	2026-05-03	양지우	요구사항 분석서 수정	
v1.5.0	2026-05-14	양지우	요구사항 정의서 최종본 제출	
v1.6.0	2026-05-16	양지우	요구사항 분석서 최종본 제출	
v1.7.0	2026-05-23	양지우	소프트웨어 설계서 작성	

목차

1. 서론
 - 1.1 문서의 목적 및 범위
 - 1.2 용어 정의
 - 1.3 참조 문서
2. 소프트웨어 아키텍처
 - 2.1 정적 구조
 - 2.2 동적 구조
3. 모듈/패키지 설계
 - 3.1 SDD-M-001 : User Package
 - 3.2 SDD-M-002 : GitHub Package
 - 3.3 SDD-M-003 : Portfolio Package
 - 3.4 패키지 행위
 - 3.5 멤버 함수 설계
4. 인터페이스 설계
 - 4.1 외부 시스템 인터페이스
 - 4.2 사용자 인터페이스
5. 데이터 설계
6. 구현 기술 설계
7. 요구사항 추적표
8. 부록

1. 서론

1.1 문서의 목적 및 범위

본 문서는 개발자 포트폴리오 웹 서비스인 GitHub-Sync 프로젝트의 소프트웨어 설계 내용을 정의하기 위해 작성되었다.

GitHub-Sync는 사용자가 GitHub 계정을 연동하여 자신의 개발 활동 데이터를 수집하고, 이를 기반으로 커밋 수, 저장소 수, 기술 스택 비율, 프로젝트 회고, 주간 목표 등을 관리할 수 있도록 지원하는 웹 기반 서비스이다.

본 문서는 시스템의 정적 구조, 동적 구조, 모듈 및 패키지 설계, 인터페이스 설계, 데이터 설계, 구현 기술 설계, 요구사항 추적표를 포함한다. 이를 통해 구현 단계에서 필요한 설계 기준을 제공하는 것을 목적으로 한다.

1.2 용어 정의

본 문서의 이해를 돕기 위해 사용된 모든 용어 및 약어를 설명하고 정의합니다.

용어	설명
GitHub	개발자가 소스코드를 저장하고 협업할 수 있는 코드 저장소 플랫폼
Repository	GitHub에서 프로젝트 소스코드를 저장하는 저장소
Commit	Git에서 파일 변경 이력을 저장하는 단위
OAuth	외부 서비스 계정을 이용하여 인증을 수행하는 방식
API	시스템 간 데이터를 주고받기 위한 인터페이스
Dashboard	데이터를 한눈에 확인할 수 있도록 시각화한 화면
Tech Stack	사용 언어 및 기술 비율 정보

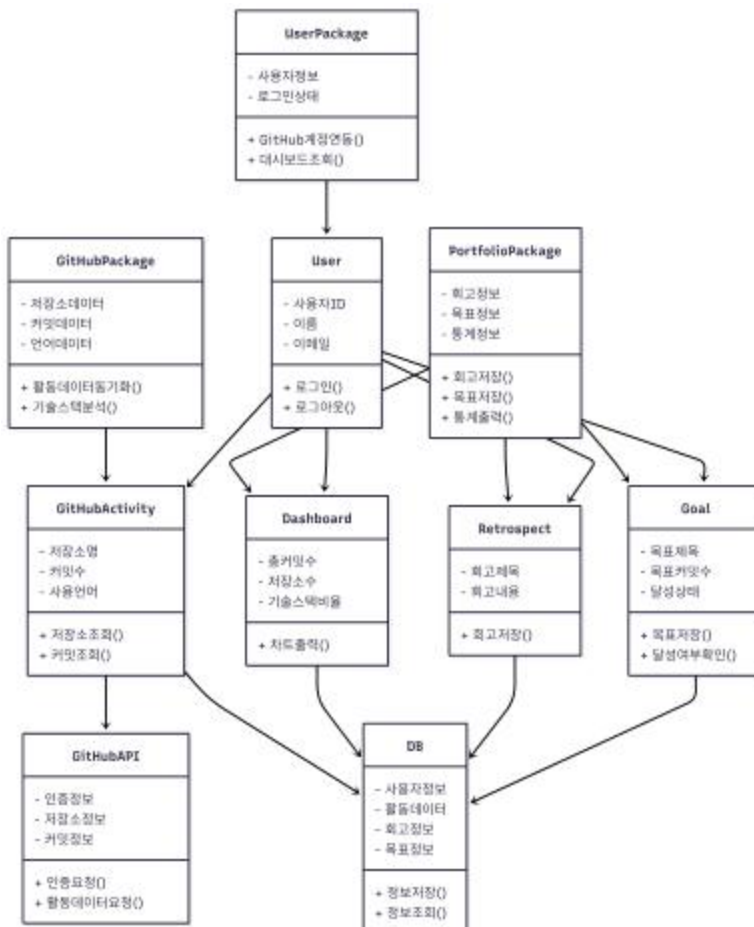
1.3 참조 문서

- [양지우]프로젝트관리계획서
- [양지우]요구사항정의서
- [양지우]요구사항분석서
- GitHub REST API Documentation
- React 공식 문서
- Node.js 공식 문서
- MySQL 공식 문서

2. 소프트웨어 아키텍처

2.1 정적 구조

2.1.1 패키지 구조

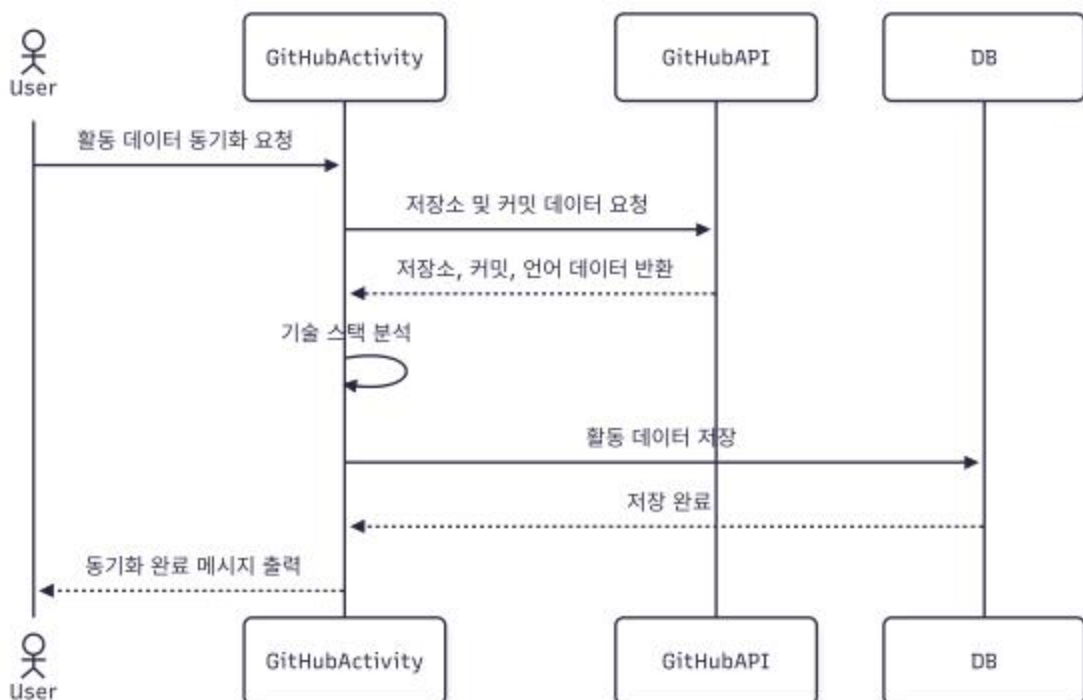


2.1.2 패키지 설명

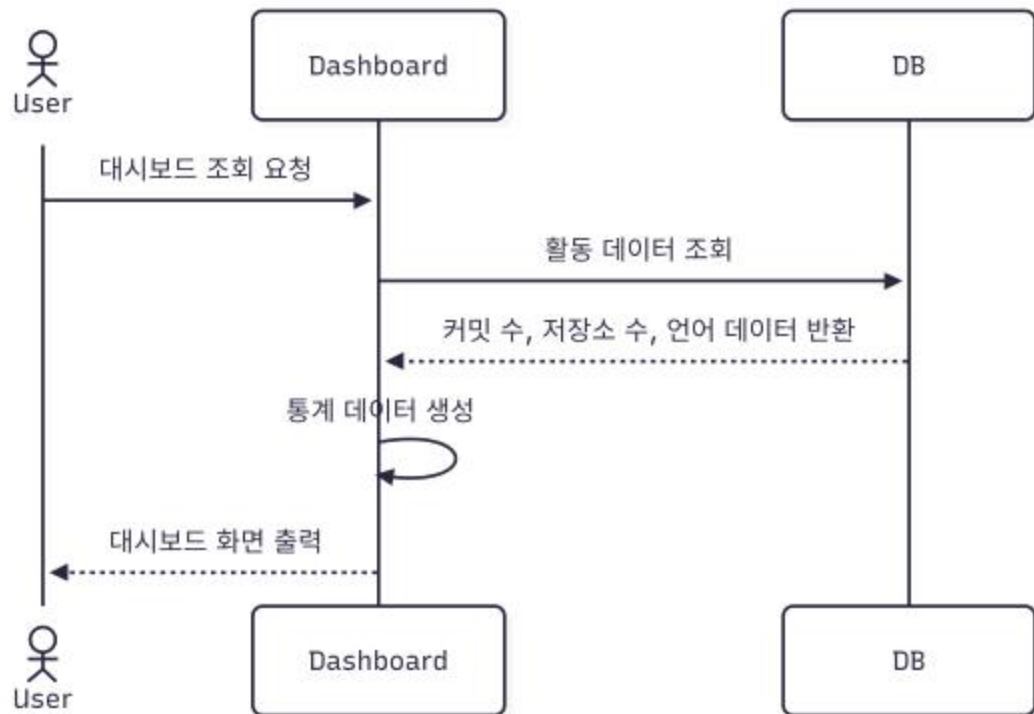
패키지명	구성 클래스	기능 설명	담당자
User Package	User	사용자의 기본 정보와 로그인 상태를 관리하며, GitHub 계정 연동과 대시보드 조회 요청을 수행한다.	양지우
GitHub Package	GitHubActivity	GitHub API를 통해 저장소, 커밋, 사용 언어 데이터를 수집하고 기술 스택을 분석한다.	양지우
Portfolio Package	Dashboard, Retrospect, Goal	수집된 개발 활동 데이터를 대시보드로 출력하고, 프로젝트 회고와 주간 목표를 관리한다.	양지우
DB	DB	사용자 정보, GitHub 활동 데이터, 회고 정보, 목표 정보를 저장하고 조회한다.	양지우
GitHubAPI	GitHubAPI	GitHub 인증 정보, 저장소 정보, 커밋 정보를 제공하는 외부 시스템이다.	외부 시스템

2.2 동적 구조

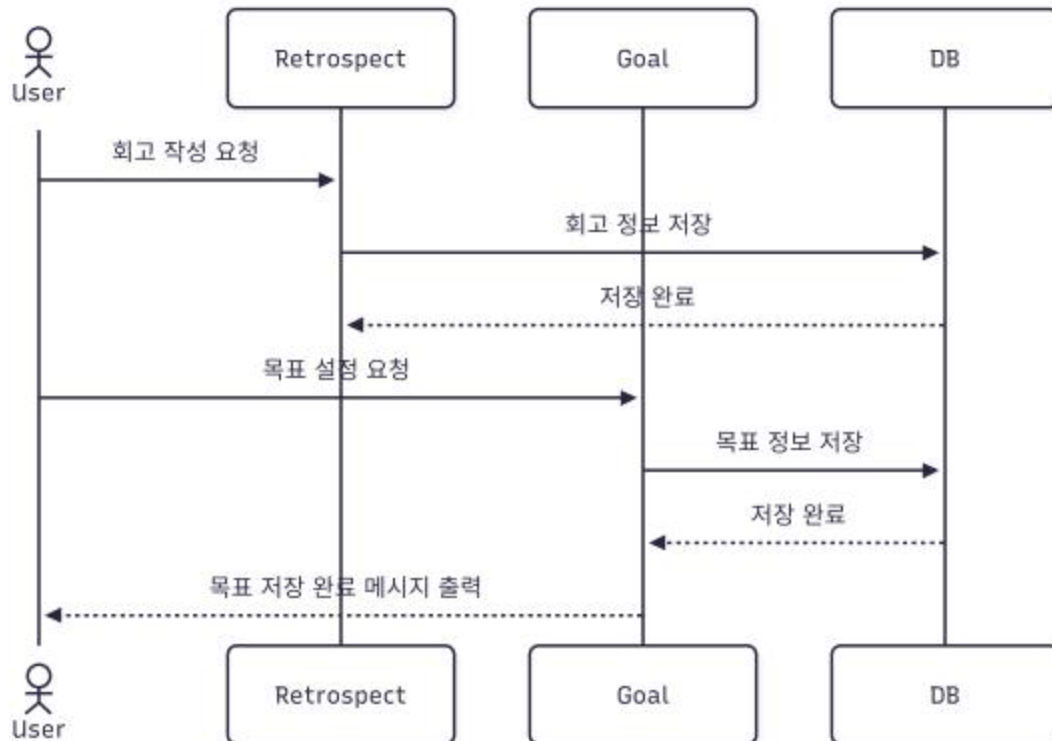
2.2.1 GitHub 활동 데이터 동기화



2.2.2 대시보드 조회



2.2.3 회고 및 목표 저장



3. 모듈/패키지 설계

3.1 SDD-M-001 : User Package

3.1.1 패키지 설명

패키지설명	사용자의 기본 정보와 로그인 상태를 관리하고, GitHub 계정 연동 및 주요 기능 이용 요청을 담당하는 패키지이다.	
구성 클래스	User	사용자의 ID, 이름, 이메일 정보를 관리하고 로그인 및 로그아웃 기능을 수행한다.

3.1.2 구성 클래스 설계

(1) CM-001 : User

클래스 타입	Concrete	관련 Use Case	U_01, U_02, U_03, U_04, U_05, U_06
설 명	GitHub-Sync 시스템을 사용하는 사용자를 나타낸다.		
멤버 변수	- 사용자ID - 이름 - 이메일		
멤버 함수	+ 로그인() + 로그아웃()		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : GitHubActivity, Dashboard, Retrospect, Goal		

3.2 SDD-M-002 : GitHub Package

3.2.1 패키지 설명

패키지설명	GitHub API와 연동하여 사용자의 저장소, 커밋, 사용 언어 데이터를 수집하고 기술 스택을 분석하는 패키지이다.	
구성 클래스	GitHubActivity	GitHub 활동 데이터 수집 및 기술 스택 분석을 담당한다.

3.2.2 구성 클래스 설계

(1) CM-002 : GitHubActivity

클래스 타입	Concrete	관련 Use Case	U_01, U_02, U_03
설 명	GitHub 저장소, 커밋, 사용 언어 데이터를 관리한다.		
멤버 변수	- 저장소명 - 커밋수 - 사용언어		
멤버 함수	+ 저장소조회() + 커밋조회() + 활동데이터동기화() + 기술스택분석()		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : User, GitHubAPI, DB, Dashboard		

3.3 SDD-M-003 : Portfolio Package

3.3.1 패키지 설명

패키지설명	사용자의 개발 활동 결과를 대시보드로 출력하고, 프로젝트 회고와 주간 목표를 관리하는 패키지이다.		
구성 클래스	Dashboard	커밋 수, 저장소 수, 기술 스택 비율을 화면에 출력한다.	
	Retrospect	사용자가 작성한 프로젝트 회고 정보를 저장한다.	
	Goal	사용자의 주간 목표와 달성 상태를 관리한다.	

3.3.2 구성 클래스 설계

(1) CM-003 : Dashboard

클래스 타입	Boundary	관련 Use Case	U_03, U_04
설 명	사용자의 개발 활동 통계를 화면에 출력한다.		
멤버 변수	- 총커밋수 - 저장소수 - 기술스택비율		
멤버 함수	+ 차트출력() + 통계출력()		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : User, GitHubActivity, Goal, DB		

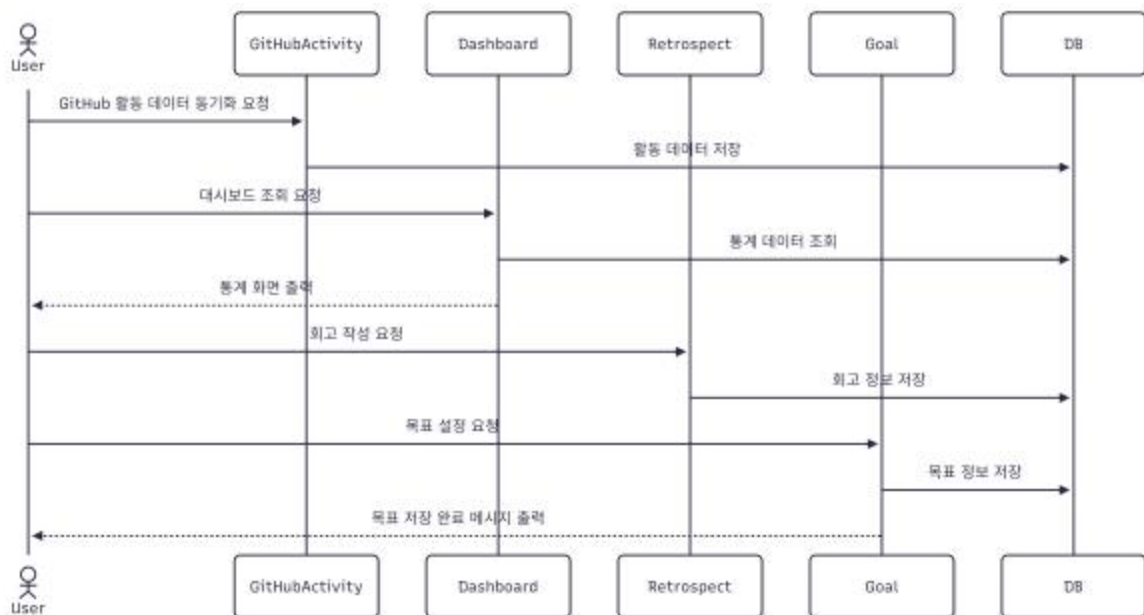
(2) CM-004 : Retrospect

클래스 타입	Concrete	관련 Use Case	U_05
설 명	사용자가 작성한 프로젝트 회고 정보를 저장하고 관리한다.		
멤버 변수	- 회고제목 - 회고내용		
멤버 함수	+ 회고저장()		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : User, DB		

(3) CM-005 : Goal

클래스 타입	Concrete	관련 Use Case	U_06
설 명	사용자의 주간 목표와 달성 상태를 관리한다.		
멤버 변수	- 목표제목 - 목표커밋수 - 달성상태		
멤버 함수	+ 목표저장() + 달성여부확인()		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : User, Dashboard, DB		

3.4 패키지 행위



3.5 멤버 함수 설계

(1) M-001 : 로그인()

소속 클래스	CM-001 : User
트리거 클래스	CM-001 : User
파라미터	이메일
세부 처리 로직	
사용자가 시스템에 접속하면 사용자 정보를 확인하고 서비스 이용 가능 상태로 전환한다.	

(2) M-002 : 활동데이터동기화()

소속 클래스	CM-002 : GitHubActivity
트리거 클래스	CM-001 : User
파라미터	사용자ID
세부 처리 로직	
GitHub API를 호출하여 사용자의 저장소, 커밋, 언어 데이터를 수집하고 DB에 저장한다.	

(3) M-003 : 기술스택분석()

소속 클래스	CM-002 : GitHubActivity
트리거 클래스	CM-003 : Dashboard
파라미터	활동 데이터
세부 처리 로직	
저장소별 사용 언어 데이터를 기반으로 언어 사용 비율을 계산한다.	

(4) M-004 : 차트출력()

소속 클래스	CM-003 : Dashboard
트리거 클래스	CM-001 : User
파라미터	사용자ID
세부 처리 로직	
DB에서 조회한 커밋 수, 저장소 수, 기술 스택 비율을 차트 형태로 출력한다.	

(5) M-005 : 회고저장()

소속 클래스	CM-004 : Retrospect
트리거 클래스	CM-001 : User
파라미터	회고제목, 회고내용
세부 처리 로직	
사용자가 입력한 프로젝트 회고 내용을 DB에 저장한다.	

(6) M-006 : 목표저장()

소속 클래스	CM-005 : Goal
트리거 클래스	CM-001 : User
파라미터	목표제목, 목표커밋수
세부 처리 로직	
사용자가 입력한 주간 목표 정보를 DB에 저장한다.	

(7) M-007 : 달성여부확인()

소속 클래스	CM-005 : Goal
트리거 클래스	CM-003 : Dashboard
파라미터	목표커밋수, 실제커밋수
세부 처리 로직	
목표 커밋 수와 실제 커밋 수를 비교하여 목표 달성 상태를 판단한다.	

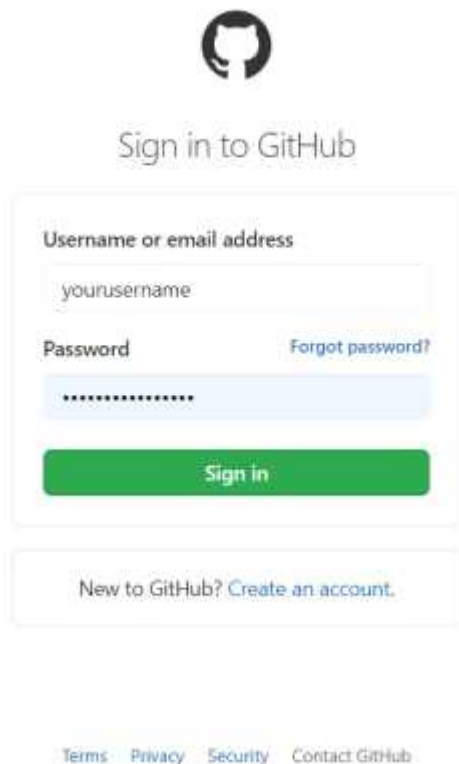
4. 인터페이스 설계

4.1 외부 시스템 인터페이스

메시지 명	송신 모듈	수신 모듈	메시지 형식	전송방식
인증 정보 요청	GitHubActivity	GitHubAPI	JSON	HTTPS
저장소 정보 요청	GitHubActivity	GitHubAPI	JSON	REST API
커밋 정보 요청	GitHubActivity	GitHubAPI	JSON	REST API
활동 데이터 저장	GitHubActivity	DB	SQL	TCP/IP
회고 정보 저장	Retrospect	DB	SQL	TCP/IP
목표 정보 저장	Goal	DB	SQL	TCP/IP

4.2 사용자 인터페이스(레퍼런스)

1. 로그인 화면



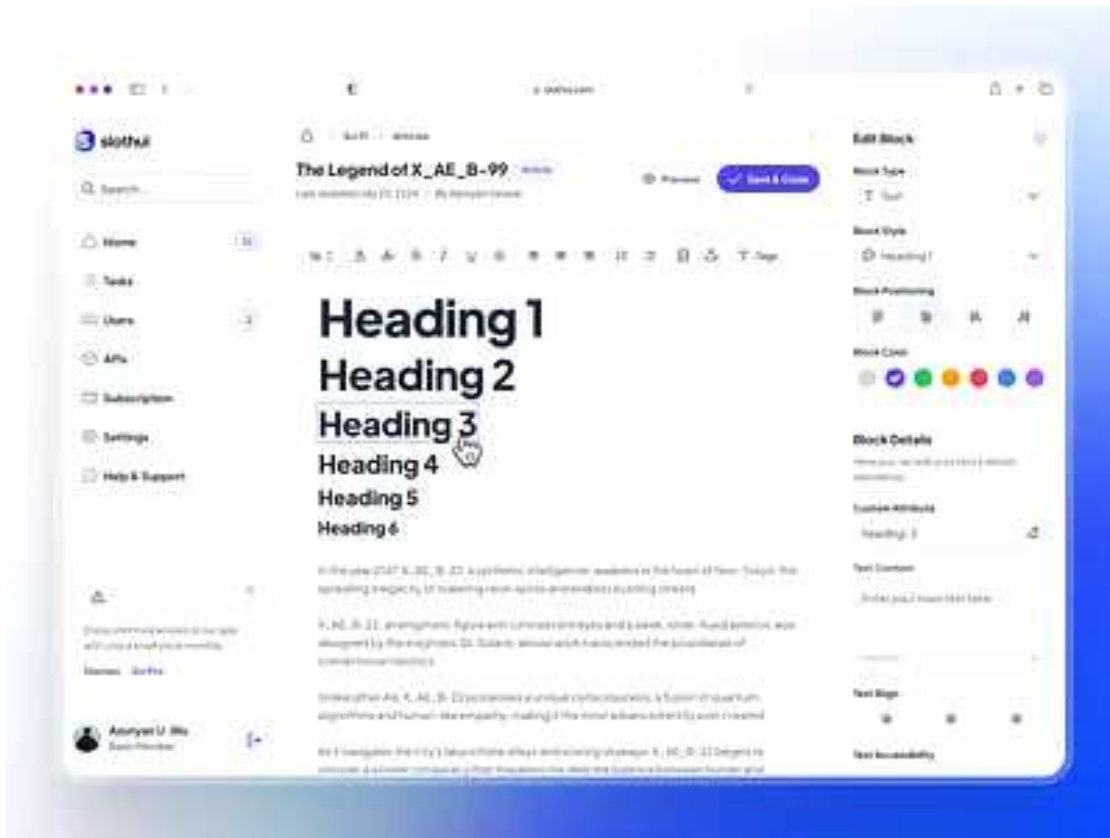
2. 대시보드 화면



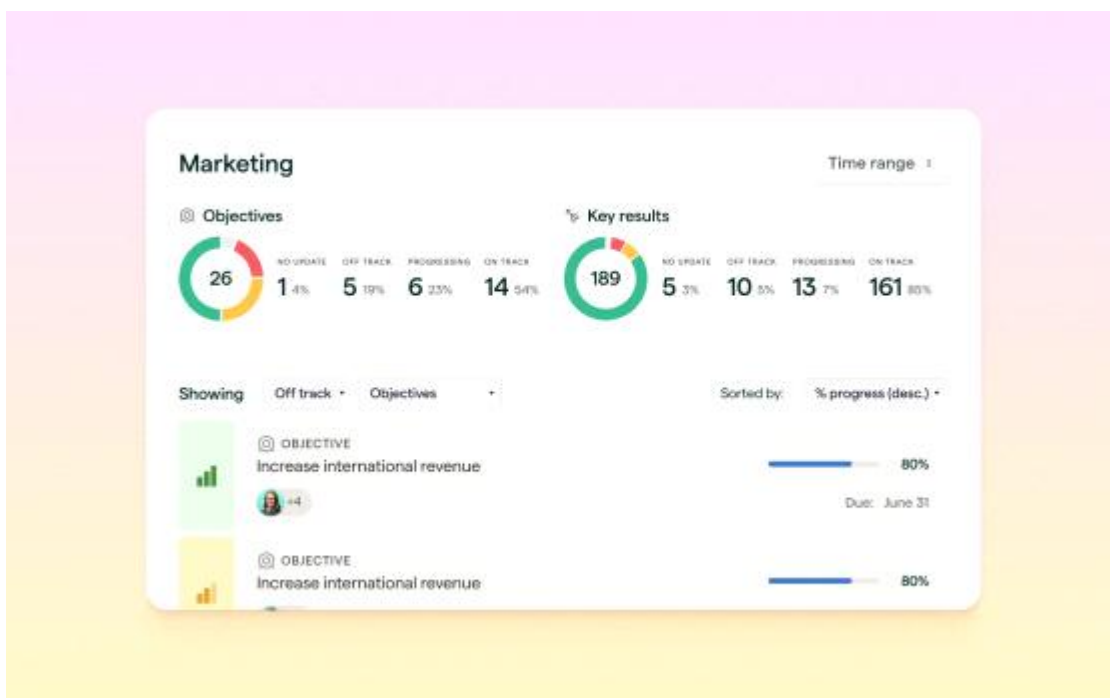
3. 기술 스택 화면



4. 회고 작성 화면



5. 목표 관리 화면



5. 데이터 설계

(1) DD-01, 사용자 정보

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	user_id	VARCHAR	NOT NULL		사용자 식별자
2	name	VARCHAR	NOT NULL		사용자 이름
3	email	VARCHAR	NOT NULL		사용자 이메일

(2) DD-02, GitHub 활동 데이터

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	activity_id	VARCHAR	NOT NULL		활동 데이터 식별자
2	user_id	VARCHAR	NOT NULL		사용자 식별자
3	repository_name	VARCHAR	NOT NULL		저장소명
4	commit_count	INT	0 이상	0	커밋 수
5	language	VARCHAR	NULL		사용 언어

(3) DD-03, 대시보드 통계 정보

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	dashboard_id	VARCHAR	NOT NULL		대시보드 식별자
2	user_id	VARCHAR	NOT NULL		사용자 식별자
3	total_commit_count	INT	0 이상	0	총 커밋 수
4	total_repository_count	INT	0 이상	0	총 저장소 수
5	tech_stack_rate	VARCHAR	NULL		기술 스택 비율

(4) DD-04, 회고 정보

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	retrospect_id	VARCHAR	NOT NULL		회고 식별자
2	user_id	VARCHAR	NOT NULL		사용자 식별자
3	title	VARCHAR	NOT NULL	0	회고 제목
4	content	TEXT	NOT NULL	0	회고 내용
5	created_at	DATE	NOT NULL	현재 날짜	작성일

(5) DD-05, 목표 정보

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	goal_id	VARCHAR	NOT NULL		목표 식별자
2	user_id	VARCHAR	NOT NULL		사용자 식별자
3	goal_title	VARCHAR	NOT NULL		목표 제목
4	target_commit_count	INT	0 이상	0	목표 커밋 수
5	status	VARCHAR	달성/미달성	미달성	달성 상태

6. 구현 기술 설계

구분	클라이언트	서버	비고
구현 언어	- JavaScript	- JavaScript	웹 서비스 구현
프레임워크	- React	- Node.js	화면 및 서버 기능 구현
데이터베이스	-	- Mysql	사용자 및 활동 데이터 저장
외부 API	- GitHub REST API	- GitHub REST API	GitHub 활동 데이터 수집
운영체제	- Windows	- Linux 또는 Windows	개발 및 실행 환경
형상관리	- Git - GitHub	- Git - GitHub	버전 관리
네트워크	- HTTPS	- HTTPS - TCP/IP	API 통신 및 DB 연결

7. 요구사항 추적표

모듈/클래스명 요구사항식별자	M-001	M-002	M-003	M-004	M-005	M-006	M-007
FR-001	●						
FR-002		●					
FR-003			●	●			
FR-004				●			●
FR-005					●		
FR-006						●	●
FR-007		●		●	●	●	●

8. 부록

- 패키지 구조도
- 동적 구조 Sequence Diagram
- 데이터 설계표
- 요구사항 추적표
- GitHub Repository 구조